

Pipes

Pipes validate the data contained within the request. To add `ValidationPipe`, for example, we have to:

```
src > TS main.ts > ...
1  import { NestFactory } from '@nestjs/core';
2  import { AppModule } from './app.module';
3  import { ValidationPipe } from '@nestjs/common';
4
5  async function bootstrap() {
6    const app = await NestFactory.create(AppModule);
7    app.useGlobalPipes(new ValidationPipe({
8      |   whitelist: true
9    }));
10   await app.listen(3000);
11 }
12 bootstrap();
```

`app.useGlobalPipes()` will apply the pipes globally in our application.

ValidationPipe

ValidationPipe allows us to use DTOs for validating the requests to our Nest application. A Data Transfer Object, or DTO, is used to describe the properties of the body, query or route parameters of the request. The format is as follows:

```
export class MethodNameDto {  
  key1: type1;  
  key2: type2;  
  ...  
}
```

The file name should be method-name.dto.ts and placed within a dtos directory inside a module directory.

To add validation to the DTO we first need to install two new packages:

```
npm i --save class-validator class-transformer
```

class-validator provides a full set of validators for the body.

class-transformer automates the process of converting json objects into javascript objects. Here, @Type() is converting the value into the Number type. Moreover, recall the whitelist: true from before, its use is to tell class-transformer to strip extra values from the body of the request that don't conform with the DTO structure. This adds extra security.

Now the MethodNameDto can be used as the type of the argument in the controller method.

An example of validating an integer can be:

```
import { Type } from "class-transformer"
import { IsInt } from "class-validator";

export class MethodNameDto {
  @Type(() => Number)
  @IsInt()
  key: number;
}
```

DTOs

The order of the decorators is important if we want to have relevant error messages. The decorators put closer to the attribute are applied first, while the ones put farther away are applied later.

We can also have nested DTOs. An example on how to achieve this:

```
import { Type } from "class-transformer"
import { ValidateNested } from "class-validator"
import { ChildDto } from "..."
```



```
export class ParentDto {
  @Type(() => ChildDto)
  @ValidateNested()
  childDto: ChildDto
}
```

`@ValidateNested()` will make Nest.js validate the nested DTO. Note that we also need to change the type.

Finally, we can make attributes optional while preserving type validation. To do so, we need to use `@IsOptional()` from `class-validator`.

ValidationPipe

The validation pipe has the following process:

1. Transform the json body into a DTO class instance using class-transformer.
2. Use the class-validator decorators to validate the DTO class instance.
3. If an error occurs, then exit immediately. Otherwise, return the body to the method argument.

ValidationPipe

Decorators for Validation and Transformation

Cuando utilizamos DTOs en **NestJS**, la validación de datos se realiza mediante decoradores provenientes de dos librerías principales:

- **class-validator** → permite validar los datos.
- **class-transformer** → convierte los datos recibidos en los tipos correctos.

Estos decoradores se colocan sobre los atributos de las clases DTO.

Validation decorators (class-validator)

DECORATOR	DESCRIPTION	EXAMPLE
@ISSTRING()	Ensures the value is a string	@IsString() name: string;
@ISINT()	Ensures the value is an integer	@IsInt() age: number;
@ISNUMBER()	Validates numeric	@IsNumber() price:

ValidationPipe

	values	number;
@ISBOOLEAN()	Validates boolean values	@IsBoolean() active: boolean;
@ISEMAIL()	Validates email format	@IsEmail() email: string;
@ISNOTEMPTY()	Ensures the field is not empty	@IsNotEmpty() title: string;
@ISOPTIONAL()	Marks the field as optional	@IsOptional() description?: string;
@LENGTH(MIN,MAX)	Restricts string length	@Length(3,20) username: string;
@MIN(VALUE)	Minimum numeric value allowed	@Min(0) stock: number;
@MAX(VALUE)	Maximum numeric	@Max(100) percent:

ValidationPipe

	value allowed	number;
@MATCHES(REGEX)	Validates using a regular expression	@Matches(/^[A-Z]+\$/) code: string;
@ISARRAY()	Ensures the value is an array	@IsArray() tags: string[];
@ARRAYNOTEMPTY()	Ensures the array is not empty	@ArrayNotEmpty() items: string[];